

Fairer Vergleich - Alle Modelle auf Iris (3 Klassen)

Datensatz: Iris, 3 Klassen, 2 Features (Index 0, 2: sepal length + petal length)

Split: test_size=0.3, random_state=42, stratify=y

Scaler: MinMaxScaler [0, 2 π]

Ziel: Alle Modelle unter identischen Bedingungen evaluieren

Modell	Typ
Quantum Kernel SVM	Quantenmodell
VQC (RealAmplitudes)	Quantenmodell
Klassischer SVM (RBF)	Klassisch
MLP (64, 32)	Klassisch

0. Imports

```
In [2]: import sys, os
sys.path.append(os.path.abspath(".."))

import numpy as np
import pandas as pd
import time
import warnings
warnings.filterwarnings("ignore")

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier

from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_machine_learning.kernels import FidelityQuantumKernel
from qiskit_machine_learning.algorithms import VQC
from qiskit.primitives import StatevectorSampler
from qiskit_machine_learning.optimizers import SPSA

from utils import save_result

print("Imports OK")
```

Imports OK

1. Datensatz - Einheitliche Pipeline

Alle Modelle verwenden exakt diesen Split und Scaler.

```
In [2]: iris = load_iris()

# Features: Index 0 (sepal length) + Index 2 (petal length)
X = iris.data[:, [0, 2]]
y = iris.target

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

# Scaler: [0, 2π] - einheitlich für alle Modelle
scaler = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

print(f"Train: {X_train_sc.shape} | Test: {X_test_sc.shape}")
print(f"Klassen: {np.unique(y_train)} | Klassenverteilung Test: {np.bincount(y_test)}
```

```
Train: (105, 2) | Test: (45, 2)
Klassen: [0 1 2] | Klassenverteilung Test: [15 15 15]
```

2. Quantum Kernel SVM

Feature Map: `zz_feature_map`, 2 Qubits, reps=2

```
In [3]: feature_map_qk = zz_feature_map(feature_dimension=2, reps=2)
kernel = FidelityQuantumKernel(feature_map=feature_map_qk)

start = time.time()
svc_q = SVC(kernel=kernel.evaluate)
svc_q.fit(X_train_sc, y_train)
train_time_qk = round(time.time() - start, 4)

start = time.time()
y_pred_qk = svc_q.predict(X_test_sc)
infer_time_qk = round(time.time() - start, 4)

acc_qk = accuracy_score(y_test, y_pred_qk)
f1_qk = f1_score(y_test, y_pred_qk, average="weighted")

print(f"Accuracy: {acc_qk:.4f} | F1: {f1_qk:.4f}")
print(f"Training: {train_time_qk}s | Inferenz: {infer_time_qk}s")
print()
print(classification_report(y_test, y_pred_qk, target_names=iris.target_names))
```

Accuracy: 0.6222 | F1: 0.6160
Training: 28.2243s | Inferenz: 23.517s

	precision	recall	f1-score	support
setosa	0.67	0.80	0.73	15
versicolor	0.64	0.60	0.62	15
virginica	0.54	0.47	0.50	15
accuracy			0.62	45
macro avg	0.62	0.62	0.62	45
weighted avg	0.62	0.62	0.62	45

3. VQC (RealAmplitudes)

Feature Map: `zz_feature_map`, reps=1 | Ansatz: `real_amplitudes`, reps=2 |
Optimizer: SPSA 150

```
In [4]: feature_map_vqc = zz_feature_map(feature_dimension=2, reps=1)
        ansatz = real_amplitudes(num_qubits=2, reps=2)

        vqc = VQC(
            feature_map=feature_map_vqc,
            ansatz=ansatz,
            optimizer=SPSA(maxiter=150),
            sampler=StatevectorSampler(),
        )

        print("Starte VQC Training...")
        start = time.time()
        vqc.fit(X_train_sc, y_train)
        train_time_vqc = round(time.time() - start, 4)

        start = time.time()
        y_pred_vqc = vqc.predict(X_test_sc)
        infer_time_vqc = round(time.time() - start, 4)

        acc_vqc = accuracy_score(y_test, y_pred_vqc)
        f1_vqc = f1_score(y_test, y_pred_vqc, average="weighted")

        print(f"Accuracy: {acc_vqc:.4f} | F1: {f1_vqc:.4f}")
        print(f"Training: {train_time_vqc}s | Inferenz: {infer_time_vqc}s")
        print()
        print(classification_report(y_test, y_pred_vqc, target_names=iris.target_names))
```

No gradient function provided, creating a gradient function. If your Sampler requires transpilation, please provide a pass manager.

Starte VQC Training...
Accuracy: 0.5333 | F1: 0.4222
Training: 261.5463s | Inferenz: 0.3237s

	precision	recall	f1-score	support
setosa	0.60	0.60	0.60	15
versicolor	0.50	1.00	0.67	15
virginica	0.00	0.00	0.00	15
accuracy			0.53	45
macro avg	0.37	0.53	0.42	45
weighted avg	0.37	0.53	0.42	45

4. Klassischer SVM (RBF-Kernel)

```
In [5]: start = time.time()
svc_rbf = SVC(kernel="rbf", random_state=42)
svc_rbf.fit(X_train_sc, y_train)
train_time_svm = round(time.time() - start, 4)

start = time.time()
y_pred_svm = svc_rbf.predict(X_test_sc)
infer_time_svm = round(time.time() - start, 4)

acc_svm = accuracy_score(y_test, y_pred_svm)
f1_svm = f1_score(y_test, y_pred_svm, average="weighted")

print(f"Accuracy: {acc_svm:.4f} | F1: {f1_svm:.4f}")
print(f"Training: {train_time_svm}s | Inferenz: {infer_time_svm}s")
print()
print(classification_report(y_test, y_pred_svm, target_names=iris.target_names))
```

Accuracy: 0.9556 | F1: 0.9556
Training: 0.0114s | Inferenz: 0.0012s

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.93	0.93	0.93	15
virginica	0.93	0.93	0.93	15
accuracy			0.96	45
macro avg	0.96	0.96	0.96	45
weighted avg	0.96	0.96	0.96	45

5. MLP (64, 32)

```
In [6]: start = time.time()
mlp = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=2000, random_state=42)
mlp.fit(X_train_sc, y_train)
train_time_mlp = round(time.time() - start, 4)
```

```

start = time.time()
y_pred_mlp = mlp.predict(X_test_sc)
infer_time_mlp = round(time.time() - start, 4)

acc_mlp = accuracy_score(y_test, y_pred_mlp)
f1_mlp = f1_score(y_test, y_pred_mlp, average="weighted")

print(f"Accuracy: {acc_mlp:.4f} | F1: {f1_mlp:.4f}")
print(f"Training: {train_time_mlp}s | Inferenz: {infer_time_mlp}s")
print()
print(classification_report(y_test, y_pred_mlp, target_names=iris.target_names))

```

```

Accuracy: 0.9333 | F1: 0.9327
Training: 0.3173s | Inferenz: 0.0012s

```

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	15
versicolor	0.83	1.00	0.91	15
virginica	1.00	0.80	0.89	15
accuracy			0.93	45
macro avg	0.94	0.93	0.93	45
weighted avg	0.94	0.93	0.93	45

6. Ergebnisse speichern

Alle 4 Modelle mit einheitlichem Datensatz-Label **Iris (3 Klassen, 2 Features, fair)**.

```

In [7]: DATENSATZ = "Iris (3 Klassen, 2 Features, fair)"

save_result("Quantum Kernel SVM", DATENSATZ, "AerSimulator", acc_qk, f1_c
save_result("VQC (RealAmplitudes)", DATENSATZ, "AerSimulator", acc_vqc, f1_v
save_result("Klassischer SVM (RBF)", DATENSATZ, "lokal", acc_svm, f1_s
save_result("MLP", DATENSATZ, "lokal", acc_mlp, f1_m

print("Alle Ergebnisse gespeichert.")

```

```

Empty DataFrame
Columns: [Quantum Kernel SVM, Iris (3 Klassen, 2 Features, fair), AerSimulator, 0.6222, 0.616, 28.2243, 23.517, zz_feature_map, 2]
Index: []
Quantum Kernel SVM Iris (3 Klassen, 2 Features, fair) AerSimulator 0.6222
0.616 28.2243 23.517 zz_feature_map 2
VQC (RealAmplitudes) Iris (3 Klassen, 2 Features, fair) AerSimulator 0.5333
0.4222 261.5463 0.3237 zz_feature_map 1
Quantum Kernel SVM Iris (3 Klassen, 2 Features, fair) AerSimulator 0.622
2 0.616 28.2243 23.517 zz_feature_map 2
VQC (RealAmplitudes) Iris (3 Klassen, 2 Features, fair) AerSimulator 0.533
3 0.4222 261.5463 0.3237 zz_feature_map 1.0
Klassischer SVM (RBF) Iris (3 Klassen, 2 Features, fair) lokal 0.955
6 0.9556 0.0114 0.0012 NaN NaN
Quantum Kernel SVM Iris (3 Klassen, 2 Features, fair) AerSimulator 0.622
2 0.616 28.2243 23.517 zz_feature_map 2
VQC (RealAmplitudes) Iris (3 Klassen, 2 Features, fair) AerSimulator 0.533
3 0.4222 261.5463 0.3237 zz_feature_map 1.0
Klassischer SVM (RBF) Iris (3 Klassen, 2 Features, fair) lokal 0.955
6 0.9556 0.0114 0.0012 NaN NaN
MLP Iris (3 Klassen, 2 Features, fair) lokal 0.933
3 0.9327 0.3173 0.0012 (64, 32) NaN
Alle Ergebnisse gespeichert.

```

7. Vergleichsübersicht

```

In [3]: df = pd.read_csv("Ergebnisse/ergebnisse.csv")
df_fair = df[df["Datensatz"].str.contains("fair")]

print(df_fair[["Modell", "Backend", "Accuracy", "F1", "Trainingszeit_s", "Inferenzzeit_s"]])

```

	Modell	Backend	Accuracy	F1	Trainingszeit_s	Inferenzzeit_s
	Quantum Kernel SVM	AerSimulator	0.6222	0.616	28.224300	23,517
	VQC (RealAmplitudes)	AerSimulator	0.5333	0.4222	261.546300	0.3237
	Klassischer SVM (RBF)	lokal	0.9556	0.9556	0.011400	0.0012
	MLP	lokal	0.9333	0.9327	0.317300	0.0012
	Quantum Kernel SVM	AerSimulator	0.8772	0.9041	408.958500	352.8897
	VQC (RealAmplitudes)	AerSimulator	0.8713	0.9027	1004.360900	1.2259
	Klassischer SVM (RBF)	lokal	0.9181	0.9381	0.007600	0.0018
	MLP	lokal	0.924	0.9412	0.914300	0.001
	VQC (RealAmplitudes, COBYLA)	AerSimulator	COBYLA	100	0.622222	0.4973921289710764
	VQC (RealAmplitudes, COBYLA)	AerSimulator	COBYLA	50	0.836257	0.8727272727272727